AI ASSISTED CODING ASSIGNMENT 19.3 2303A52490

Task 1: Python to C++ Conversion

Original Python class for Student:

```python
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}")

# Test
s = Student("John Doe", 20, "A")
s.display()
```

Translated C++ code (student.cpp):

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int age;
    string grade;

public:
    Student(string n, int a, string g) : name(n), age(a), grade(g) {}

    void display() {
        cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << endl;
    }
};

int main() {
    Student s("John Doe", 20, "A");
    s.display();
    return 0;
```

```
}
```

Compilation and Execution (Terminal Output):

```
$ g++ student.cpp -o student
$ ./student
Name: John Doe, Age: 20, Grade: A
```

Python output:

```
$ python3 student.py
Name: John Doe, Age: 20, Grade: A
```

Both outputs match perfectly. Handled constructor init list, private members, and string types.

Task 2: Java to Python Function Conversion

Original Java method:

```
public class PrimeChecker {
  public static boolean isPrime(int num) {
    if (num <= 1) return false;
    for (int i = 2; i <= Math.sqrt(num); i++) {
      if (num % i == 0) return false;
    }
    return true;
  }
}
```

Translated Python function (prime.py):

```
import math

def is_prime(num):
  if num <= 1:
    return False
  for i in range(2, int(math.sqrt(num)) + 1):
    if num % i == 0:
      return False
  return True

# Tests
```

```
test_numbers = [2, 17, 15, 1, 29, 4]
for n in test_numbers:
    print(f"{n}: {is_prime(n)}")
```

Terminal Output:

```
$ python3 prime.py
2: True
17: True
15: False
1: False
29: True
4: False
```

Verified with multiple inputs. Used range and math.sqrt for Pythonic style.

Task 3: Pseudocode to Python Implementation

Pseudocode for Bubble Sort:

```
ALGORITHM BubbleSort(A)
    FOR i from 0 to length(A)-1
        FOR j from 0 to length(A)-i-1
            IF A[j] > A[j+1]
                SWAP A[j] and A[j+1]
            END IF
        END FOR
    END FOR
END ALGORITHM
```

Translated Python (bubble.py):

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Test
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Original:", numbers)
bubble_sort(numbers)
```

```
print("Sorted:", numbers)
```

Terminal Output:

```
$ python3 bubble.py
Original: [64, 34, 25, 12, 22, 11, 90]
Sorted: [11, 12, 22, 25, 34, 64, 90]
```

Validated with sample input. Swaps implemented directly

Task 4: SQL to Pandas Query

Sample SQL query:

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Sample DataFrame setup and Pandas code (pandas_query.py):

```
import pandas as pd

data = {
   'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
   'salary': [60000, 45000, 70000, 30000]
}
df = pd.DataFrame(data)

# Equivalent Pandas query
result = df[df['salary'] > 50000][['name', 'salary']]
print(result)
```

Terminal Output:

```
$ python3 pandas_query.py
     name  salary
0   Alice   60000
2  Charlie   70000
```

Filters correctly matching SQL. Used boolean indexing

Task 5: Real-Time Application Algorithm Translation

Selected: Python Binary Search to C.

Original Python (binary_search.py):

```python
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Test
arr = [1, 3, 5, 7, 9]
print(binary_search(arr, 5))  # Output: 2
```

Translated C (binary_search.c):

```c
#include <stdio.h>

int binary_search(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int arr[] = {1, 3, 5, 7, 9};
    int n = 5;
    printf("%d\n", binary_search(arr, n, 5));  // Output: 2
    return 0;
}
```

Terminal Output for C:

```
$ gcc binary_search.c -o binary
$ ./binary
2
```

Challenges Faced:

- C needs array size explicitly; Python uses len().
- Integer division same, but manual array handling in C.
- No built-in lists; used fixed array.

Both versions return same index for tests.